David Gabrick

Mechanical Engineering Portfolio

Mechanical Engineering student at the University of Texas at Arlington, focused on building practical, real-world systems: design and fabrication of mechanical components, vision-guided control, and high-performance vehicle systems with UTA Racing Formula SAE.

EMAIL
davidgabrick.dg@gmail.com

LINKEDIN
linkedin.com/in/david-gabrick

PORTFOLIO
davidgabrick.github.io

Contents

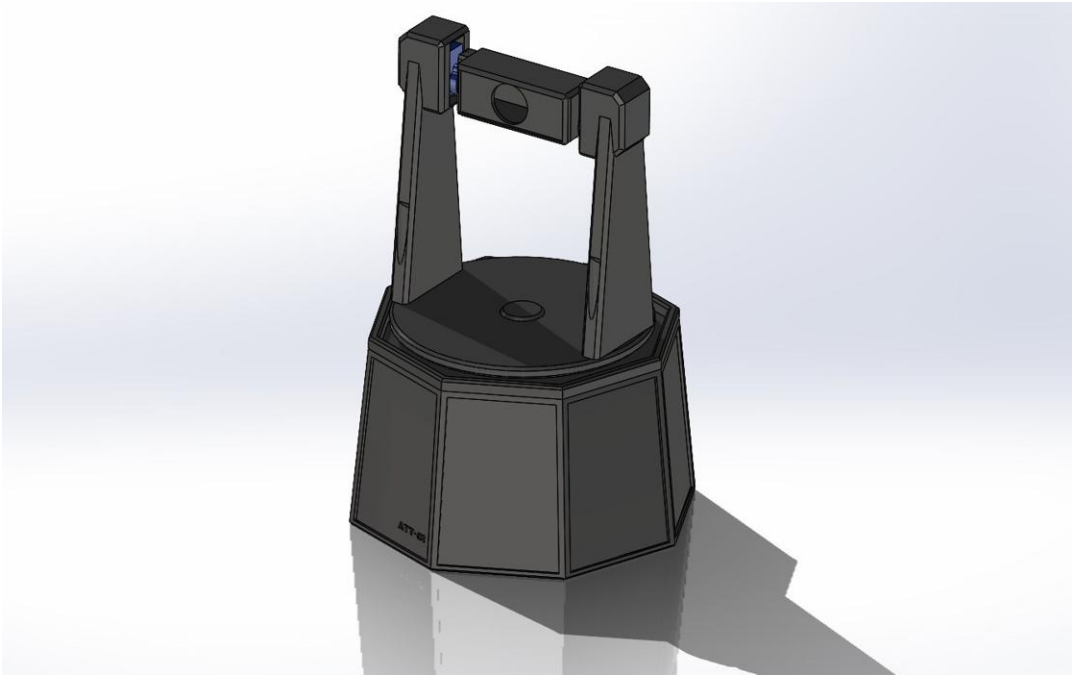
About

I'm a Mechanical Engineering student at the University of Texas at Arlington with a focus on building practical, real-world systems. I have experience designing and fabricating mechanical components and currently contribute to the UTA Racing Formula SAE team developing high-performance vehicle systems.

The two projects in this portfolio cover both halves of that work. The Autonomous Turret Tracker is a solo, full-stack vision-guided control platform: mechanical design, electronics, firmware, perception, and a closed loop tuned and characterized from logged data. The Level-Luffing Mechanical Crane is a four-person academic design project I led, taking a linkage mechanism from concept through SolidWorks, FEA, and a drawing package to a working 3D-printed prototype. The full engineering report for each is reproduced as an appendix.

Autonomous Turret Tracker (ATT-01)

A vision-guided pan-tilt platform that detects a person, holds lock, and keeps the target centered in real time.



Overview

ATT-01 is a self-contained pan-tilt platform that keeps a moving target centered in frame. A laptop detects a person with a YOLOv8n neural network, holds a persistent lock on that specific subject with ByteTrack, and estimates the target's motion with a Kalman filter. A per-axis PID loop turns the tracking error into servo-angle commands, which stream over Wi-Fi to an ESP32 that drives the pan and tilt servos. What separates it from a typical hobby tracker is that it was tuned and characterized like a real control system, with every design decision backed by logged data.

TYPE	TIMEFRAME	TEAM SIZE	MY ROLE
Personal project	**Summer 2026**	**Solo**	**Whole stack: mechanical, electronics, firmware, vision, control**

Why I built it

I want to work on vision-guided control systems in defense and aerospace. The interest started with photography and a curiosity about how stabilized camera gimbals hold a subject locked in frame. ATT-01 is my hands-on route into that problem: the same detection, estimation, and closed-loop control that keep a gimbal locked on its subject.

The Objective

The aim was a project that shows real systems-engineering work, not just a quick demo. I set out to detect and follow a specific person in real time and hold lock even when they stop moving; to close a genuine feedback loop from camera to servo and tune it with engineering rigor instead of trial and error; to instrument the system so every decision could be backed by measured data; and to integrate the full stack, from optics and vision through estimation, control, firmware, and a 3D-printed mechanical assembly, into one working platform.

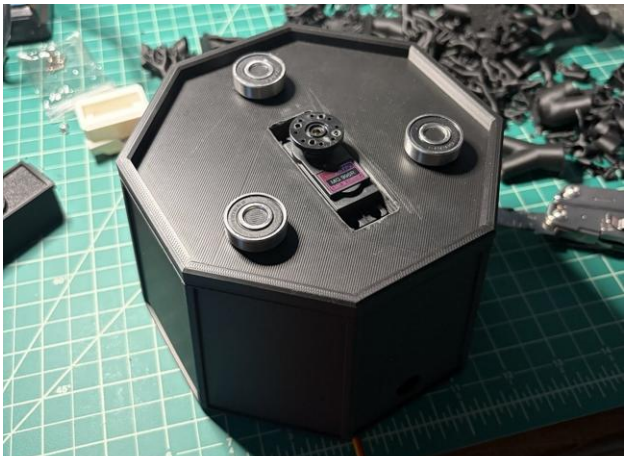
Approach & Process

System architecture

The platform splits the work between a host laptop, which handles perception and control, and an ESP32, which handles actuation. The two talk over Wi-Fi. Keeping the heavy vision and control math on the laptop let me iterate quickly while the microcontroller stayed responsible for one job: driving the servos.

```
USB webcam → YOLOv8n detection → ByteTrack ID → centroid → Kalman filter → per-axis PID → UDP over Wi-Fi → ESP32 → 50 Hz PWM → MG996R (pan) + SG90 (tilt)
```

Section render of the pan and tilt drivetrain.



3D-printed pedestal base with internal hardware.

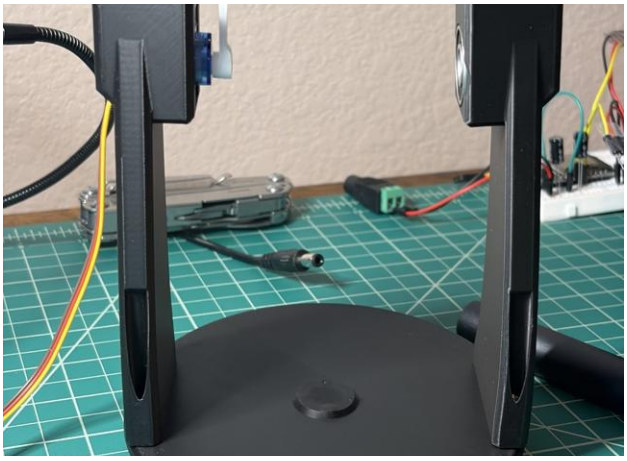
Mechanical design

The pan-tilt assembly is a custom 3D-printed bracket. The pan axis uses an MG996R servo to swing the whole camera mass; the tilt axis uses a lighter SG90. Because the two axes are physically different plants with different inertia, I treated and tuned them separately rather than forcing one set of gains on both. The full paid bill of materials came to about $45.

Detection and locking

Instead of reacting to motion, the platform detects a designated target. YOLOv8n classifies a person in each frame, and ByteTrack assigns a persistent ID so the system keeps following the same subject through clutter, a second mover, and brief occlusion, and does not lose them when they stand still. After a sustained loss, it reacquires on its own.

Camera module mounted on the tilt deck.



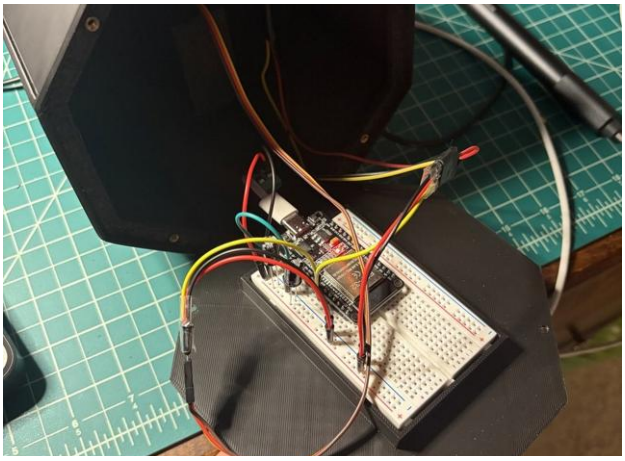
Assembled pan-tilt deck and camera arms.

State estimation and control

A constant-velocity Kalman filter fuses a motion model with the noisy detection. That rejects bounding-box jitter and lets the platform coast through short detection dropouts. An independent PID per axis turns the filtered pixel error into incremental angle commands. The loop carries several guards I added during tuning: a slew-rate limit that caps how far the command can move in a single frame, so a spike becomes slow, correctable motion instead of a full-speed swing; derivative-kick protection on target acquisition; and a deadband sized just above one command step so the platform locks cleanly instead of dithering.

Electronics and firmware

Commands travel over UDP for low-latency, fire-and-forget delivery. The ESP32 firmware, written in C++ on the Arduino framework, receives the angle targets and drives both servos by PWM. The servos run from a dedicated 5 V / 3 A supply sized to the MG996R stall current, sharing a common ground with the ESP32 (never its 3.3 V pin), with flyback diodes and decoupling capacitors for power integrity.



Wiring: ESP32, servo supply, and power-integrity components.

Closing the Loop: Six Failures, Found in Data

The most instructive part of the project was closing the loop. Every piece passed its bench test in open loop, but the moment the camera was bolted on and started reacting to its own motion, a sequence of control failures showed up that an open-loop test simply cannot reveal. I found each one in logged data, traced it to a root cause, fixed it, and verified the fix.

1. **Derivative kick.** On acquisition the error jumped from zero to full in one frame, spiking the command and slamming the pan servo into its stop. I seeded the derivative history, clamped the loop timestep, and added a slew-rate limit.
2. **Inverted feedback sign.** The loop pushed the subject out of frame instead of centering them. I corrected the pan direction, changing one variable at a time so every result stayed attributable.
3. **Delay-induced limit cycle.** A stationary subject produced a steady oscillation of about 400 px at roughly 0.77 Hz that never decayed. A non-decaying oscillation at fixed gain is the Ziegler-Nichols ultimate-gain condition, so the fault itself handed me the tuning constants ($K_u \approx 0.04$, $T_u \approx 1.3$ s). I backed the gain below K_u and added derivative damping.
4. **Derivative noise.** The derivative term amplified bounding-box jitter into servo motion, worse the farther away the subject stood. I reduced derivative action and low-pass filtered the centroid error.
5. **Kalman ego-motion regression.** Adding predictive lead made tracking worse, because the filter read the camera's own panning as target velocity and chased a ghost. I damped the velocity estimate and disabled the lead until camera motion could be compensated.
6. **Hidden resolution bug.** The camera was delivering 1280 px frames, not the 640 px the gains assumed, which doubled every error and held the command permanently at the slew-rate limit, so every frame moved by the maximum allowed step. That is bang-bang behavior, and bang-bang is gain-independent, which is why no gain change could fix it. Forcing 640×480 cut slew saturation from 68% of frames to 1%, and steady-state error from 333 to 58 px RMS.

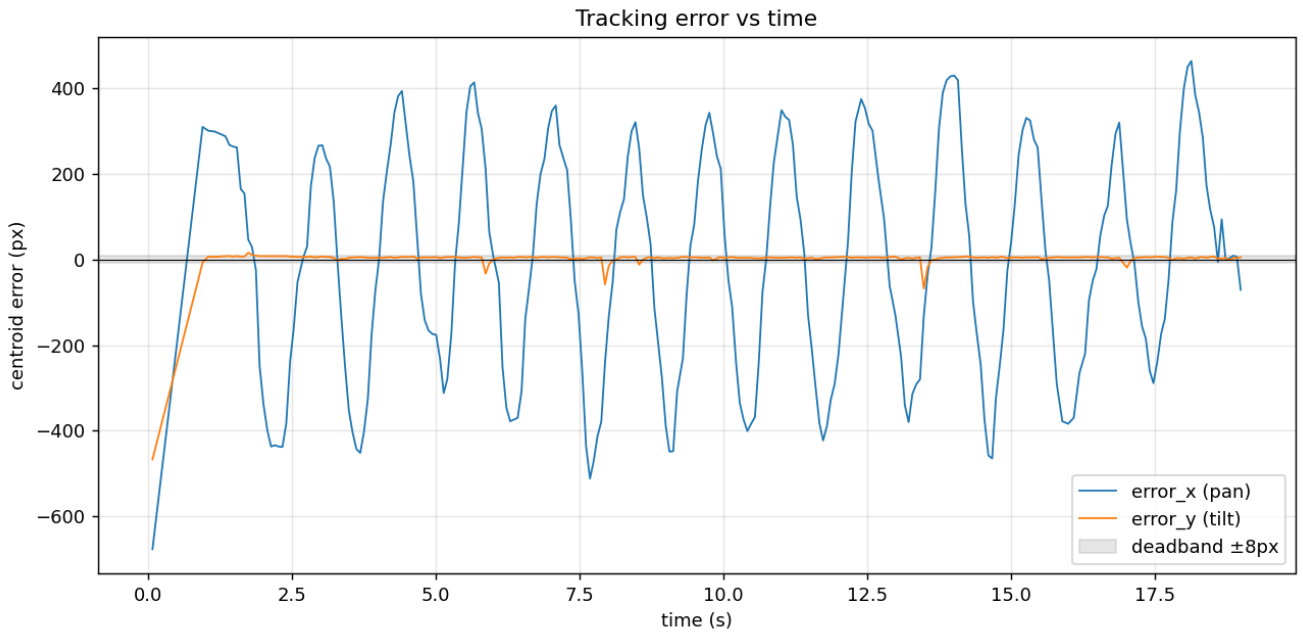
One humbling detail: the tilt servo was unplugged for the entire tuning history, so every "tilt is stable" result had been an artifact of a disconnected actuator. Once connected, it showed the same inverted-sign fault and was corrected the same way. The throughline across all of it was to trust the logged data over my own assumptions and to look upstream of the obvious suspect before reaching for a more complex fix.

Tools & Skills

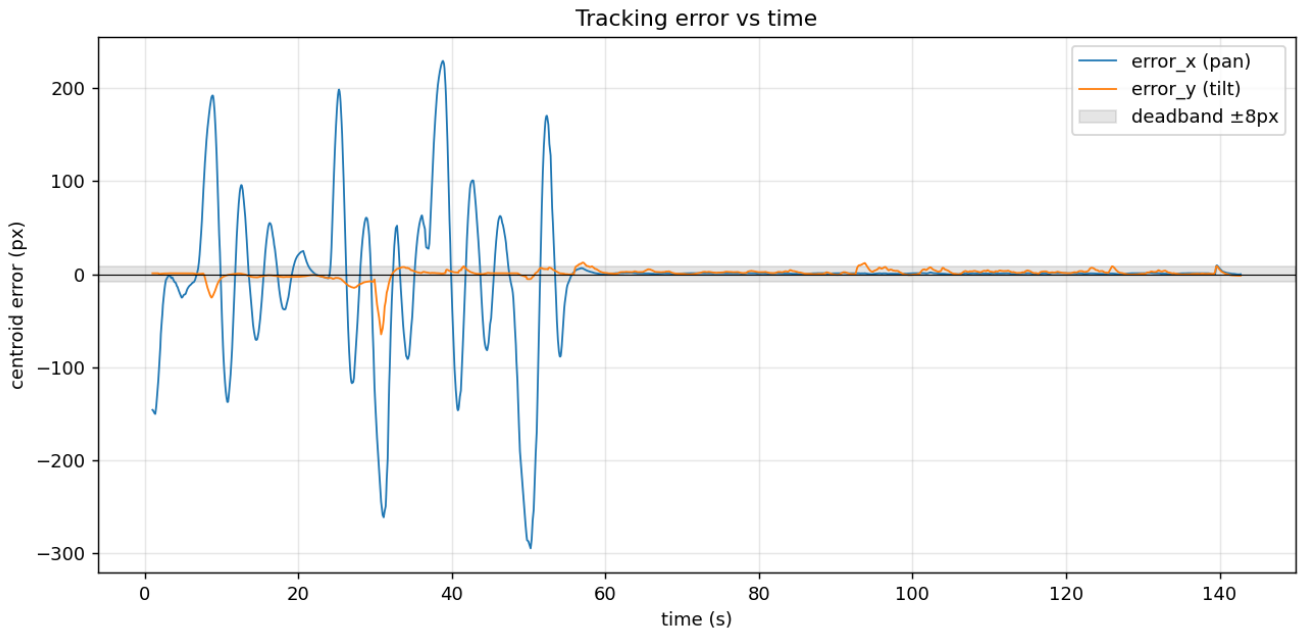
Python · YOLOv8n · ByteTrack · OpenCV · Kalman filtering · PID control · Ziegler-Nichols tuning · Control characterization · Data logging and analysis · ESP32 · Arduino / C++ · UDP over Wi-Fi · PWM servo control · Power electronics · Fusion 360 · 3D printing (PLA)

Results & Performance

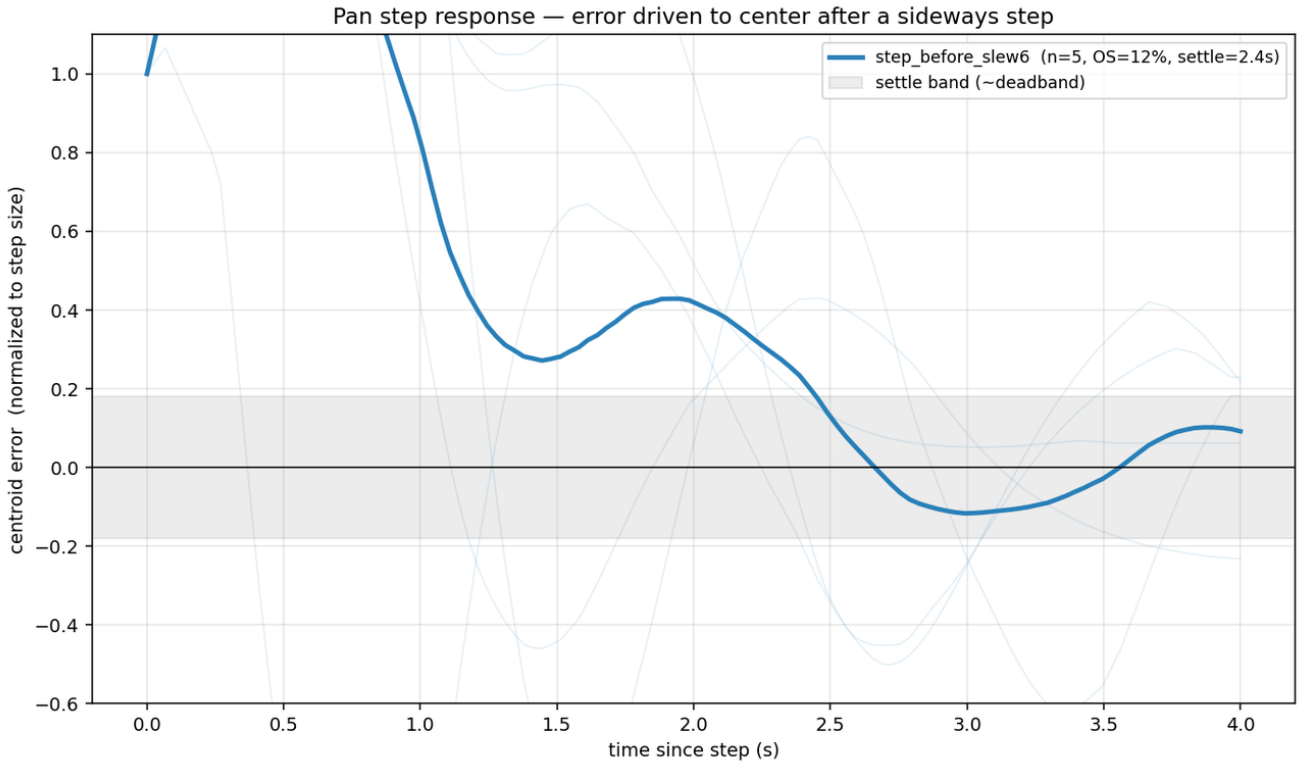
The tuned loop holds lock on nearly every frame, settles cleanly when the subject stops, and has a measured step response. The control story is clearest in three figures: an uncontrolled oscillation before tuning, a stable lock after, and a step response that characterizes the tuned loop.



Before tuning: pan (blue) holds a constant-amplitude limit cycle that never settles; tilt (orange) sits flat.



After the resolution and saturation fix: both axes settle into the deadband once the subject stops moving.



Pan step response across repeated sideways-step tests. The bold line is the mean: about 12% overshoot, 1.6 s rise, and 2.4 s settling, all from measured data.

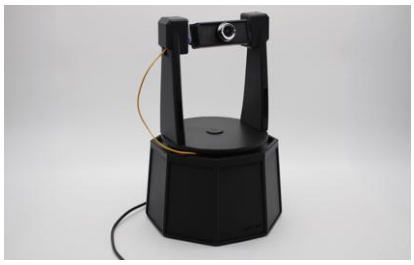
Measured performance

Target lock rate	**99–100% of frames**
Control-loop rate	**≈ 15 Hz**
Steady-state error (pan)	**40–55 px RMS**
Step response (pan)	**12% overshoot, 1.6 s rise, 2.4 s settle**
Slew saturation (resolution fix)	**68% → 1% of frames**
Steady-state error (resolution fix)	**333 → 58 px RMS**

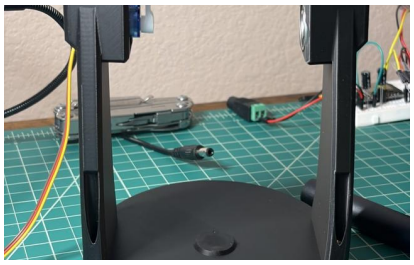
One result is worth stating plainly: the remaining overshoot does not respond to further gain or filter tuning, because the loop is now delay-limited. Two controlled experiments, one varying the derivative gain and one varying the Kalman smoothing, left it essentially unchanged. That is a characterization result, not a shortcoming, and it points to the next upgrade: lower latency or a predictor.

Live demo video. A recording of the turret detecting and tracking a target in real time is embedded on the portfolio site: davidgabrick.github.io

Gallery

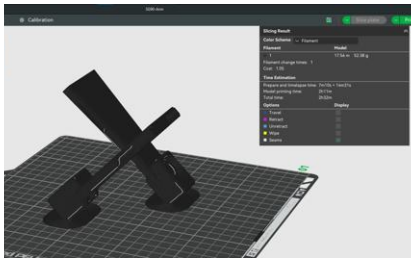


The finished build



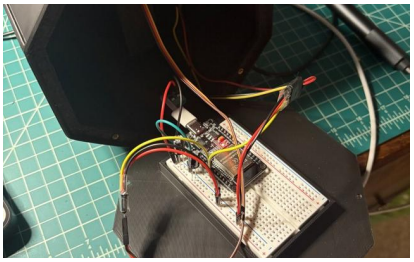
Assembled pan-tilt deck

Section render of the drivetrain



Parts staged in the slicer

Pedestal base



Electronics and wiring

Documentation

ATT-01 Engineering Case Study — reproduced in full as Appendix A (9 pages)

Bill of Materials (XLSX) — available on request, or at davidgabrick.github.io

The case study covers the architecture, the control design, all six closed-loop failures and fixes, and the measured results, with every figure reproduced from logged run data.

Level-Luffing Mechanical Crane

The White Crane

Overview

"The White Crane" was our final design project for MAE 1351, Introduction to Engineering Design (Dr. Raul Fernandez). Working in a four-person team, we designed, simulated, and fabricated a **level-luffing crane**: a crane whose linkage keeps its end effector level as the jib reaches in and out, instead of letting it rise and dip through the arc. We designed it in SolidWorks, ran an FEA study, and built a working 3D-printed prototype driven by a linear actuator.

TYPE	TIMEFRAME	TEAM SIZE	MY ROLE
Academic team project (MAE 1351)	**Spring 2026**	**4 (Team 2, "The White Crane")**	**Project Lead, most of the design and build**

My contribution

I led the four-person team and did most of the design and build work. The level-luffing concept came from research I did during brainstorming, which we then adapted for our needs. I modeled every part in SolidWorks (the base frame, linkage arms, claw, actuator mounts, and the brackets and spacers), which kept the assembly consistent and let me catch fitment problems before anything was printed. I also produced the working drawings, including the assembly drawing and the detail drawings with tolerances and GD&T. I printed every part on a Bambu Lab printer (reprinting the ones that came out weak or didn't fit), assembled the full mechanism (linkages, actuator, rack and pinion, and claw), soldered the wiring for the servos, actuator, battery, and controls, and helped write the control code that sequenced the actuator and claw.

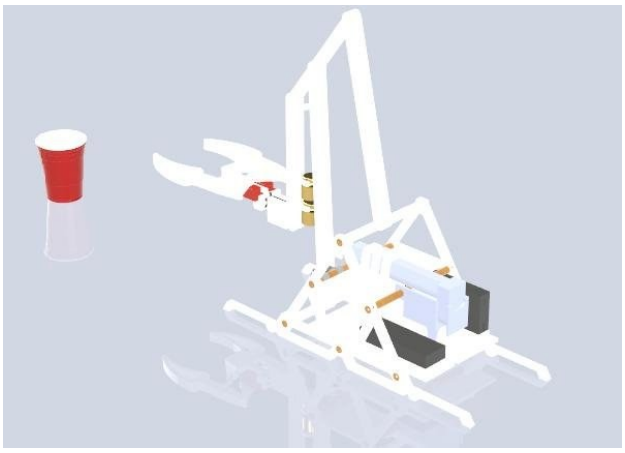
The Challenge

The graded task was to flip a Solo cup autonomously. A standoff rule made it harder: the mechanism had to stay at least one foot away from the cup, so the crane had to reach in from a distance and flip the cup rather than work right next to it. Flipping the cup once within 30 seconds earned full credit. We went further: the crane flipped the cup once, then twice more in a row to return the cup to its original position, which earned bonus points.

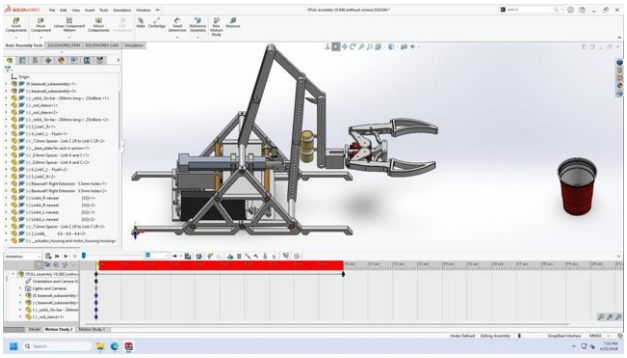
Approach & Process

Keeping the claw level

Our main modification to the standard level-luffing design was letting the front (reaching) arm dangle, or pivot freely, as it extends forward. Extending the arm to lower the claw had been making the claw clip into the ground; allowing the arm to pivot keeps the claw level through the reach instead.



The reaching arm and leveling claw linkage.



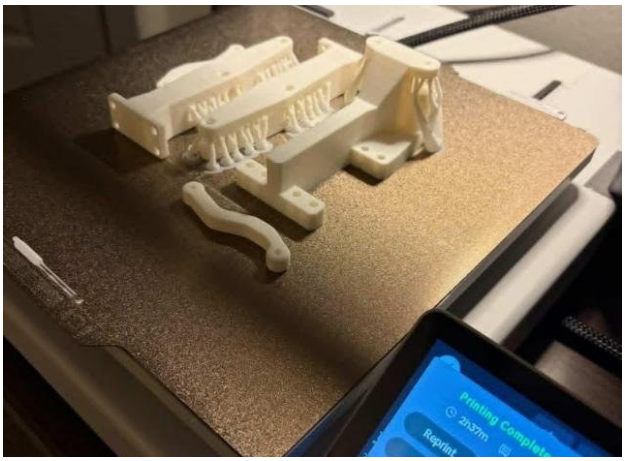
Assembly view: open base frame and stabilizing legs.

Stability without a counterweight

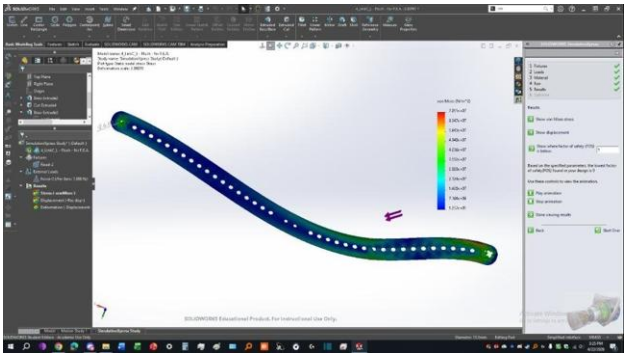
Rather than balance the crane with a counterweight, we removed it entirely and added legs to the base for stability. We also reworked the base into a more open, more cost-effective frame.

Fabrication, actuation & controls

Every part was 3D-printed on a Bambu Lab printer, with weak or ill-fitting parts reprinted, then assembled into the working mechanism: linkages, the linear actuator, a rack and pinion, and the claw. The claw's grabbers are fastened with four 6 mm M4 threaded inserts (a commercial off-the-shelf kit) for durable machine-screw threads. The servos, actuator, battery, and controls were wired and soldered in, and control code sequenced the actuator and claw so they moved in the right order.



A printed part on the Bambu Lab bed.


Von Mises stress plot of the actuated link at 1000 N.

Simulation & FEA

We analyzed the link the actuator pushes on using SolidWorks Simulation. The load was applied at the 11th hole from the bottom-right, where the linear actuator acts on the link, with fixtures at the top-left (the connecting link) and the bottom-right (fixed to the base's rod). At the actuator's maximum output of 1000 N, the Von Mises results showed most of the part in low stress, with a localized high-stress region at the fixed bottom-right area. That spot is the most likely to deform and the clearest candidate for reinforcement in a future revision.

Tools & Skills

SolidWorks (CAD) · SolidWorks Visualize · SolidWorks Simulation (FEA) · Engineering drawings and GD&T · Mechanism and linkage design · 3D printing (Bambu Lab) · Linear actuator and rack-and-pinion · Wiring and soldering · Control programming · Threaded-insert assembly · Project leadership

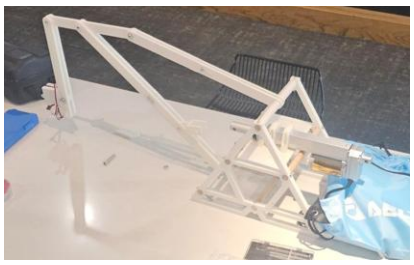
Results & Outcomes

The prototype did the job: it flipped the Solo cup autonomously within the time limit for full credit, then flipped it twice more in a row to return the cup to its original position for bonus points. The dangling-arm modification kept the claw level through the reach, and the legged base provided stability in place of a

counterweight. The SolidWorks FEA confirmed the actuated link carries the actuator's 1000 N maximum load with stress concentrated at the fixed end, the spot we'd reinforce in a future revision.

Gallery

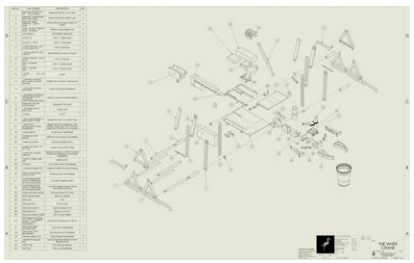
SolidWorks Visualize render



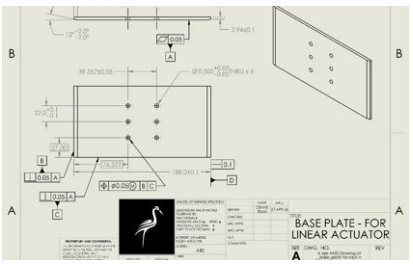
The completed prototype

Lifting with the claw kept level

Mid-lift, second angle



Assembly drawing



Detail drawing (tolerances & GD&T)

Documentation

White Crane Design Report — reproduced in full as Appendix B (10 pages)

Covers the overview, design rationale, the FEA study, and the engineering drawing package. The roster page listing teammates' student ID numbers has been removed for privacy.

In Progress

Cycloidal Drive Gearbox (Fall 2026) — a high-reduction cycloidal gearbox. Personal project, not yet started; design, tooling, and results will be added to the portfolio site as the work progresses.

UTA Racing — Formula SAE — ongoing contribution to high-performance vehicle systems with the UTA Racing Formula SAE team.

Live portfolio: davidgabrick.github.io — includes the ATT-01 tracking demo video, full-resolution images, and downloadable documentation.
Contact: davidgabrick.dg@gmail.com · linkedin.com/in/david-gabrick

APPENDIX A

ATT-01 Engineering Case Study

The complete 9-page case study for the Autonomous Turret Tracker: system architecture, control design, all six closed-loop failures and their fixes, and the measured results, with every figure reproduced from logged run data.

ATT-01

Autonomous Target-Tracking Platform

(ATT = Autonomous Target Tracker)

An Engineering Case Study

Real-time vision-guided target tracking: computer vision, state estimation, closed-loop control, and embedded systems

David Gabrick
Mechanical Engineering, University of Texas at Arlington
Summer 2026

Executive Summary

ATT-01 (Autonomous Target Tracker) is a self-contained pan-tilt platform that detects a designated target with a deep-learning object detector, estimates its motion with a Kalman filter, and drives two servos through a closed control loop to keep the target centered in frame. A laptop runs the computer-vision and control pipeline and streams angle commands over Wi-Fi to an ESP32 microcontroller that drives the servos.

Many hobby projects can track a target. What sets this one apart is that it was engineered and characterized like a real control system. During closed-loop bring-up, the platform exposed six distinct control failures in sequence: a derivative kick, two inverted feedback signs, a Ziegler-Nichols-characterized limit cycle, a Kalman ego-motion regression, and a camera-resolution-induced slew saturation. Each one was found in logged data, traced to a root cause, fixed, and verified. The final loop is stable, holds lock on 99 to 100 percent of frames, and is delay-limited, a conclusion backed by measured step-response data.

The finished system brings together embedded firmware, computer vision, state estimation, control theory, and mechanical design in one platform. It is accompanied by a written record of how the system was debugged and validated.

1. Motivation and Objectives

The goal was to build a portfolio project that demonstrates end-to-end systems-engineering capability for roles in defense and aerospace, where vision-guided actuation, real-time control, and embedded integration are directly relevant.

The project set out to:

- Detect and follow a specific target (a person) in real time, holding lock even when the target stops moving.
- Close a real feedback loop from camera to actuator, and tune it with engineering rigor rather than trial and error.
- Instrument the system so every design decision could be justified with measured data.
- Integrate the full stack, from optics and vision through estimation, control, firmware, and a 3D-printed mechanical assembly, into one working platform.

> *I built this because I want to work on vision-guided control systems in defense and aerospace, the kind of work done at Lockheed Martin, Northrop Grumman, L3Harris, and Bell. My interest started with photography, where I got curious about how stabilized camera gimbals hold a shot steady and keep a subject locked in frame. This platform is my hands-on way into that problem: the same detection, estimation, and closed-loop control that keep a gimbal locked on its subject.*

2. System Architecture

The platform splits work between a host laptop (perception and control) and an ESP32 microcontroller (actuation), connected over Wi-Fi:

USB webcam > YOLOv8n detection > ByteTrack persistent ID > raw centroid > Kalman filter (smoothed position and velocity) > per-axis PID > UDP over Wi-Fi > ESP32 > 50 Hz servo PWM > MG996R (pan) and SG90 (tilt).

Key design choices

- UDP over Wi-Fi for low-latency, fire-and-forget host-to-microcontroller commands.
- Detector-based tracking (YOLOv8n plus ByteTrack) instead of motion subtraction, so the platform follows a designated target through clutter and brief occlusion and does not lose a stationary subject.
- A dedicated 5 V / 3 A servo supply with common ground (never the microcontroller's 3.3 V pin), flyback diodes, and decoupling capacitors, with power integrity sized to the MG996R stall current.
- A custom 3D-printed pan-tilt bracket. Total paid bill of materials is about $45.

3. Control System Design

The host computes the pixel error between the target's estimated position and frame center, and an independent PID controller per axis converts that error into incremental servo-angle commands. Several features make the loop robust:

- Independent per-axis gains. Pan (an MG996R swinging the whole camera mass) and tilt (a light SG90) are physically different plants with different inertia, so they are tuned separately rather than sharing one gain set.
- Constant-velocity Kalman filter. The PID is driven by a filtered state estimate rather than the raw, jittery detection. Fusing a motion model with the measurement rejects bounding-box noise and lets the platform coast through brief detection dropouts.
- Slew-rate limit. The per-frame command is capped so the commanded angle cannot change faster than a set step, which bounds command velocity and inertial overshoot.
- Derivative-kick protection. The derivative term seeds its history on the first frame after acquisition, so the step from zero to full error cannot spike the command.
- Deadband sized to the actuator. The rest zone is set just above one command-quantization step so the platform locks confidently instead of dithering.

4. Bringing the Loop to Life: Six Failures, Diagnosed from Data

The most instructive part of the project was closed-loop bring-up. Before assembly, the software chain passed every bench test, but the servos were unmounted, so the camera never moved in response to its own commands. That is an open loop. Once the camera was bolted on and the loop truly closed, a sequence of faults appeared that an open-loop test cannot reveal. Each one is presented below as symptom, root cause, and fix.

4.1 Servo slams to its end stop on target acquisition

Symptom: The instant the detector locked on, the pan servo snapped about 90 degrees into a hard stop.

Root cause: A derivative kick. On acquisition the error steps from 0 to its full value in one frame, and the derivative term divides that step by the tiny frame time, producing a one-frame command spike of about 59 degrees that overwhelms everything else. Flipping the feedback sign, the obvious first guess, produced no change, which correctly ruled the sign out.

Fix: Seed the derivative's history on the first frame after acquisition so its computed rate is zero, clamp the loop timestep, and add a hard slew-rate limit. The slew limit also turned later faults from violent blurs into slow, readable motion that could be diagnosed.

4.2 Tracker drives the target out of frame

Symptom: With the slam gone, the camera panned so as to push the subject out of frame, then ran to its travel limit.

Root cause: An inverted feedback sign for the as-built camera orientation. The result is positive feedback, so error grew instead of shrinking. This is the sign the open-loop bench test could not have validated.

Fix: Correct the pan direction. One variable changed at a time, leaving the tilt axis untouched so every result stayed attributable.

4.3 Sustained oscillation when the subject stands still

Symptom: With direction correct, a stationary subject produced a clean, constant-amplitude pan oscillation (about plus or minus 400 px at roughly 0.77 Hz) that never decayed.

Root cause: A delay-induced limit cycle: loop transport delay plus a gain above the stability threshold. A non-decaying oscillation at a fixed gain is the Ziegler-Nichols ultimate-gain condition, so the failure itself yielded the tuning constants (Ku about 0.04, Tu about 1.3 s). Only pan oscillated, because it carries far more rotational inertia than the light tilt axis.

Fix: Back the gain below Ku and add derivative damping, a Ziegler-Nichols-informed PD design suited to a delay-dominated plant.

4.4 Distance-dependent hunting

Symptom: After further tuning, pan hunted more the farther away the subject stood, and appeared inactive up close.

Root cause: The derivative term amplifying detector noise. A smaller, farther target has a noisier bounding box, and the D term converted that jitter into full-rate servo motion. Up close,

the box is clipped by the frame edges, so its centroid pins near center and the controller correctly rests. That is real behavior, not a fault.

Fix: Reduce derivative action and low-pass the centroid error before it reaches the controller. This is standard practice for vision servoing, and it set up the Kalman filter that followed.

4.5 Kalman filter regression: the ego-motion lesson

Symptom: Adding a Kalman filter with predictive lead made tracking worse, not better. Steady-state error roughly tripled.

Root cause: Ego-motion. In a closed loop the measured centroid also moves when the camera pans, so the filter read the camera's own motion as target velocity. A stationary subject logged hundreds of px/s of phantom velocity. The predictive lead then projected that fiction forward into the command, which is positive feedback. An offline open-loop test did not reproduce it, and that is what located the cause in the closed loop.

Fix: Damp the velocity estimate hard and disable predictive lead until camera motion is compensated. The filter then serves as a position smoother and a coast-through-dropout mechanism, verified to roughly halve measurement noise.

4.6 The hidden variable: camera resolution drove slew saturation

Symptom: A stationary subject still produced a large, slow oscillation, and reducing the gain, even to a quarter of Ku, never helped.

Root cause: The camera was delivering a frame about 1280 px wide, not the roughly 640 px assumed when sizing the gains. That doubled every pixel error and held the command permanently at the slew-rate limit, producing a bang-bang oscillation. Bang-bang is gain-independent, which is why lowering the gain did nothing. The detected-frame error reached plus or minus 692 px, impossible inside a 640 px frame, which exposed the true resolution.

Fix: Force the capture to 640x480 and use a single-frame buffer. Maximum error halved, the command stayed under the slew limit, and the loop became linear. Slew saturation dropped from 68 percent of frames to 1 percent, and steady-state error fell from 333 px to 58 px RMS.

One practical note: the tilt servo had been physically unplugged for the entire tuning history, so every earlier observation that tilt was stable was an artifact of a disconnected actuator. Once connected, tilt showed the same inverted-sign fault as Issue 2 and was corrected the same way.

5. Results and Performance

The control story is clearest in three figures: an uncontrolled oscillation before tuning, a stable lock after, and a measured step response that characterizes the tuned loop.

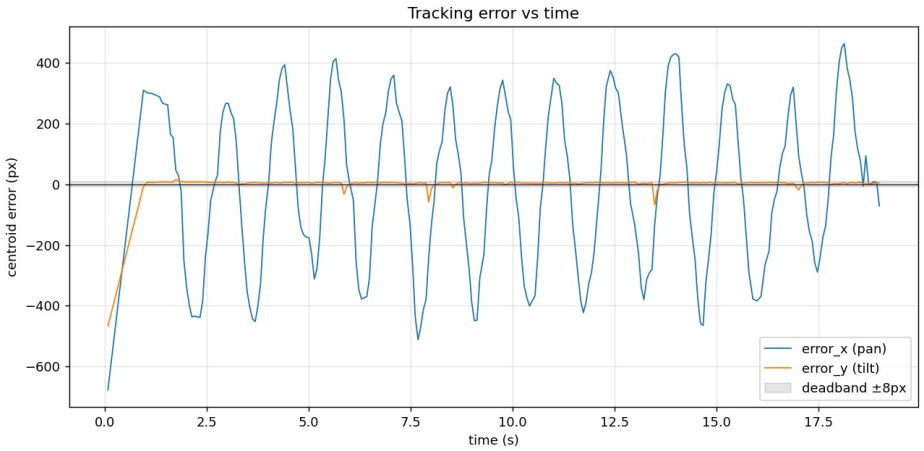


Figure 1. Before tuning: pan (blue) is a constant-amplitude limit cycle that never settles, while tilt (orange) sits flat.

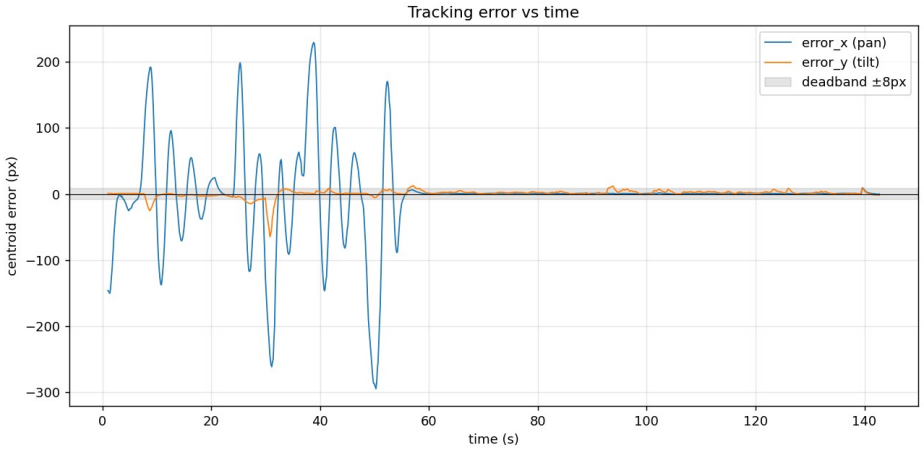


Figure 2. After the resolution and saturation fix: both axes settle into the deadband once the subject stops moving.

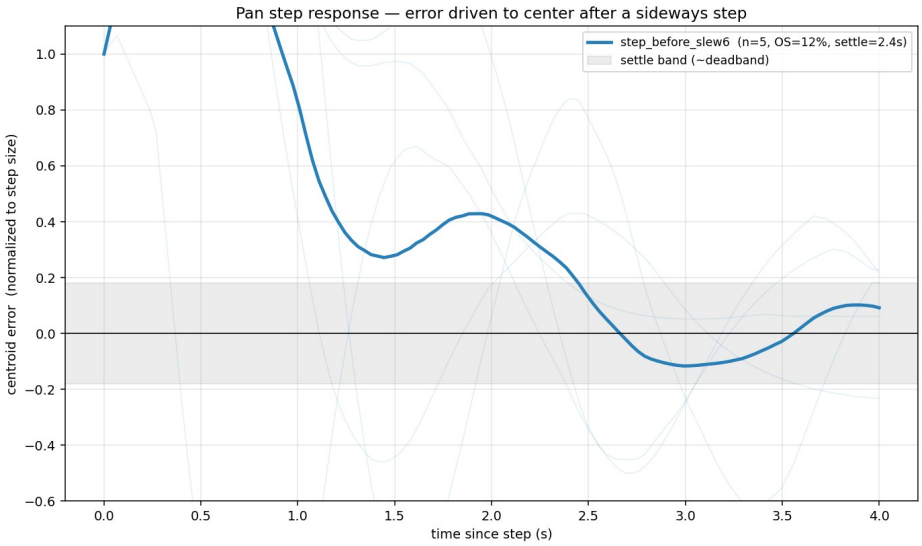


Figure 3. Ensemble pan step response from sideways-step tests. Faint traces are individual steps and the bold line is the mean: about 12 percent overshoot, 1.6 s rise, and 2.4 s settling, taken from measured data.

Measured performance

Metric	Value
Target lock rate	99 to 100 percent of frames
Control-loop rate	about 15 Hz (YOLOv8n, 480 px inference, laptop)
Steady-state tracking error (pan)	about 40 to 55 px RMS
Step response (pan)	about 12 percent overshoot, 1.6 s rise, 2.4 s settle
Ziegler-Nichols constants (pan)	Ku about 0.04, Tu about 1.3 s
Slew saturation (1280 px to 640 px fix)	68 percent to 1 percent of frames
Steady-state error (1280 px to 640 px fix)	333 px to 58 px RMS

One conclusion is worth stating plainly: the residual step overshoot does not respond to further gain or filter tuning, because the loop is now delay-limited. Two controlled experiments, one varying the derivative gain and one varying the Kalman smoothing, left the overshoot essentially unchanged, which isolates the cause to loop latency and servo inertia. This is a characterization result rather than a shortcoming, and it points to the upgrade that would help next: lower latency or a predictor.

6. Lessons Learned

1. A passing open-loop test proves the data path, not stability. Every closed-loop fault was invisible on the bench.
2. A negative result is information. Flipping the sign and seeing no change is what ruled it out and pointed at the real cause.
3. Make failures observable before interpreting them. The slew-rate limit did not fix the sign error, but it slowed the motion enough to diagnose it.
4. Let the system reveal its own constants. The standstill oscillation doubled as a Ziegler-Nichols experiment that handed over Ku and Tu directly.

5. Sensor configuration is part of the control loop. Resolution, frame rate, and buffering set loop gain and delay as surely as the controller gains do.
6. A model-based estimator is only as good as its model. The Kalman filter's constant-velocity model was right for the target but blind to the camera's own motion.

> *The most surprising moment was adding a Kalman filter and watching tracking get worse instead of better, until I saw it was reading the camera's own panning as target motion and chasing a ghost. The hardest was the oscillation that would not go away no matter how far I dropped the gain, which turned out to be the camera quietly feeding me a 1280 px frame instead of the 640 px I had sized everything around. And the most humbling was realizing the tilt servo had been unplugged the whole time, so every result I had trusted for that axis meant nothing. Across all three, the lesson was the same: trust the logged data over my own assumptions, and look upstream of the obvious suspect before reaching for a more complex fix.*

7. Future Work

- Lower latency. Move inference to a GPU or Jetson, or use a smaller model, to push the loop past 15 Hz. Because the overshoot is delay-limited, halving latency roughly halves overshoot.
- Sub-degree actuation. Command the servos in fractional degrees (microsecond PWM) to roughly triple angular resolution and shrink the deadband without re-introducing dither.
- Ego-motion-compensated prediction. Subtract the camera's commanded angular rate from the estimated target velocity, then re-enable predictive lead to anticipate motion instead of reacting to it.
- Mechanical analysis. Servo-torque and inertia calculations, plus finite-element analysis on the bracket, to formalize the mechanical design.

Appendix A. Final Configuration

Parameter	Pan	Tilt
Proportional gain (Kp)	0.018	0.020
Derivative gain (Kd)	0.008	0.004
Integral gain (Ki)	0	0
Deadband	22 px (about 2 deg)	22 px
Slew limit	4 deg per frame	4 deg per frame

Subsystem	Detail
Detector	YOLOv8n (person class), 480 px inference
Tracker	ByteTrack persistent ID
Estimator	Constant-velocity Kalman filter (position smoother and coast)
Capture	640x480, single-frame buffer
Link	UDP over 2.4 GHz Wi-Fi to ESP32
Actuators	MG996R (pan), SG90 (tilt), 5 V / 3 A supply

A full record of every fault and fix is kept in the project's control-systems tuning log. All figures are reproduced from logged run data using the project's own plotting tools.

APPENDIX B

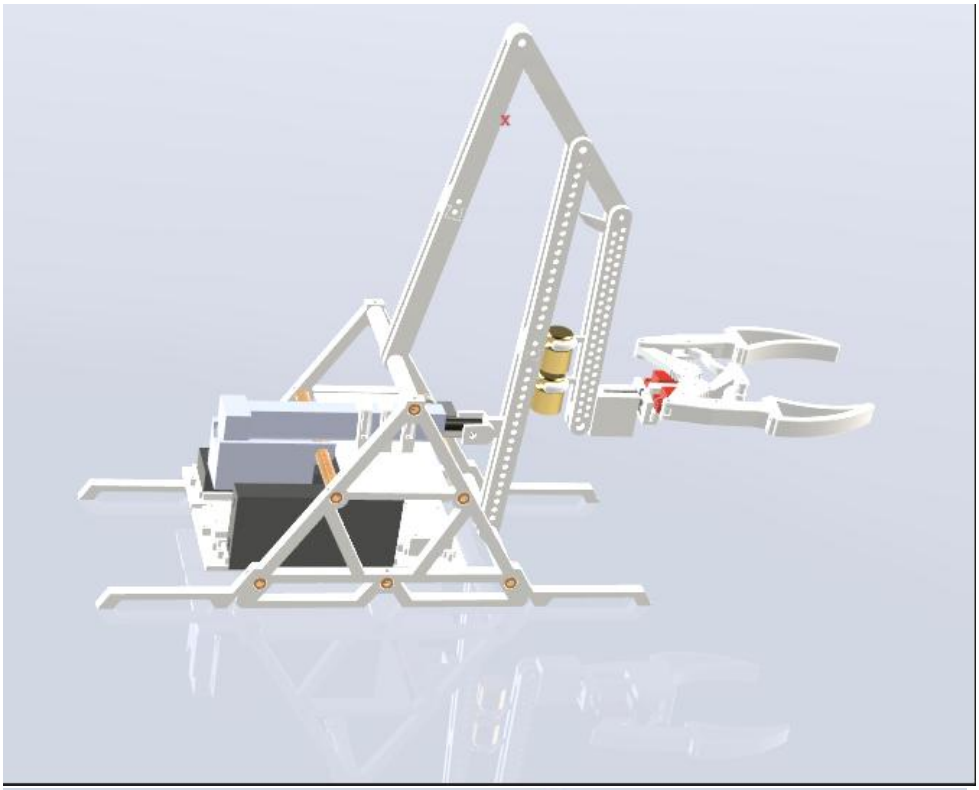
White Crane Design Report

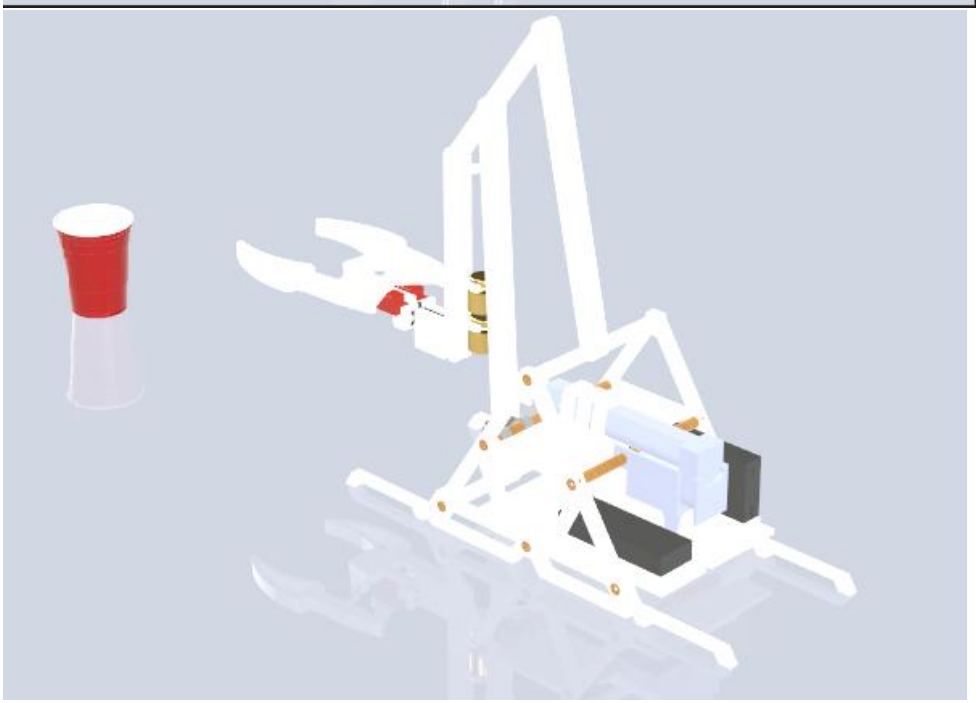
The complete 10-page design report for the Level-Luffing Mechanical Crane: project overview, design rationale, the SolidWorks FEA study, and the full engineering drawing package. The roster page listing teammates' student ID numbers has been removed for privacy.

1. OVERVIEW

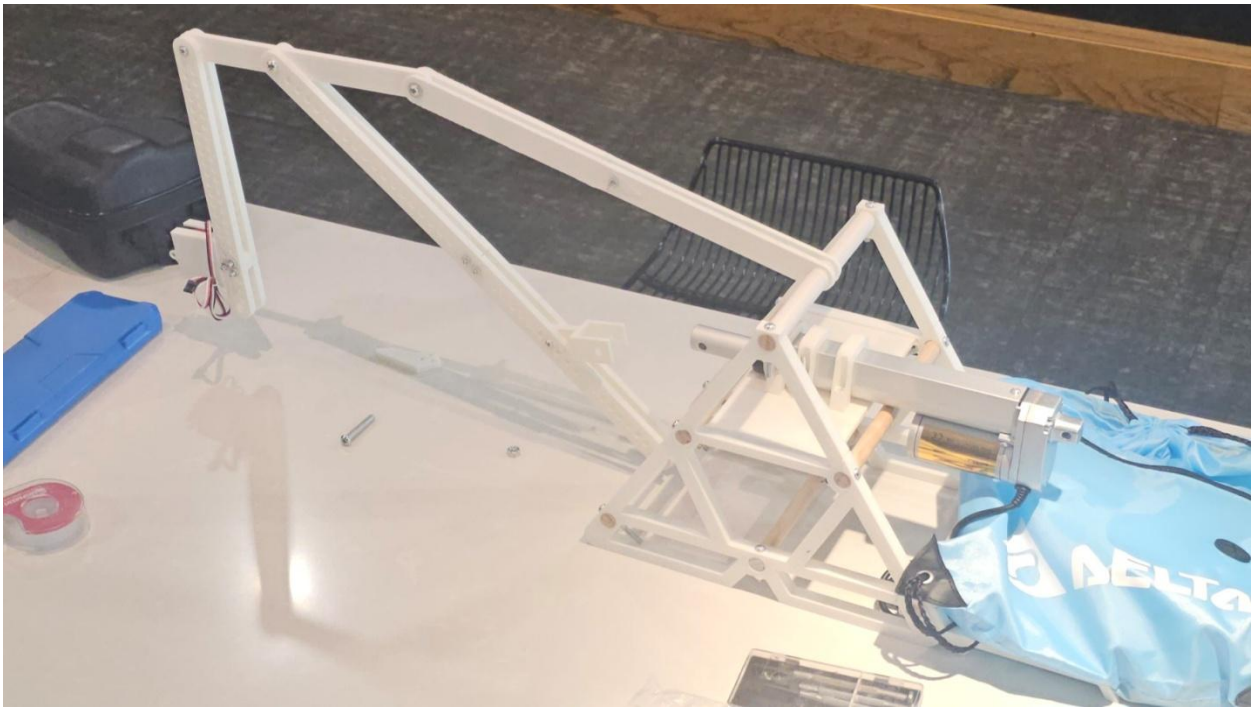
1.1. Device images

SolidWorks Visualize:



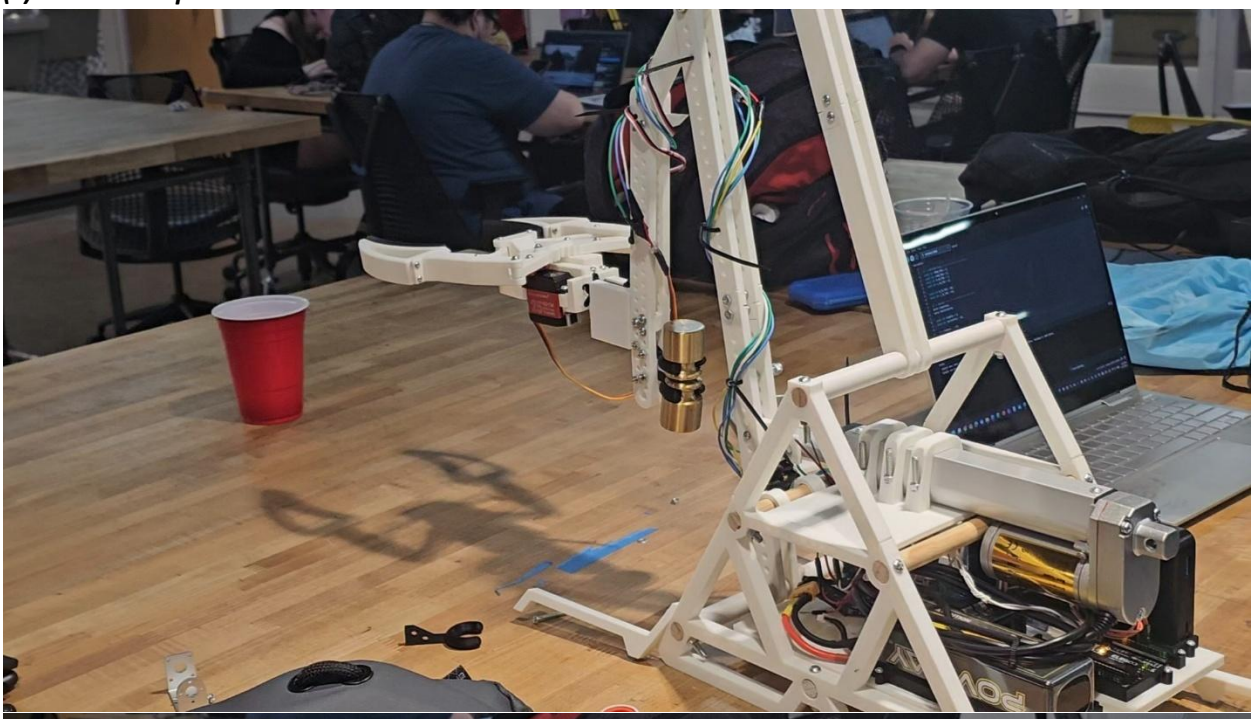


(a) Overall View of Prototype:



(b) One Part Being Fabricated

(c) Device in Operation.



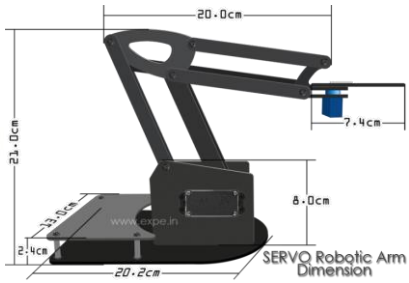
1.2. Design references

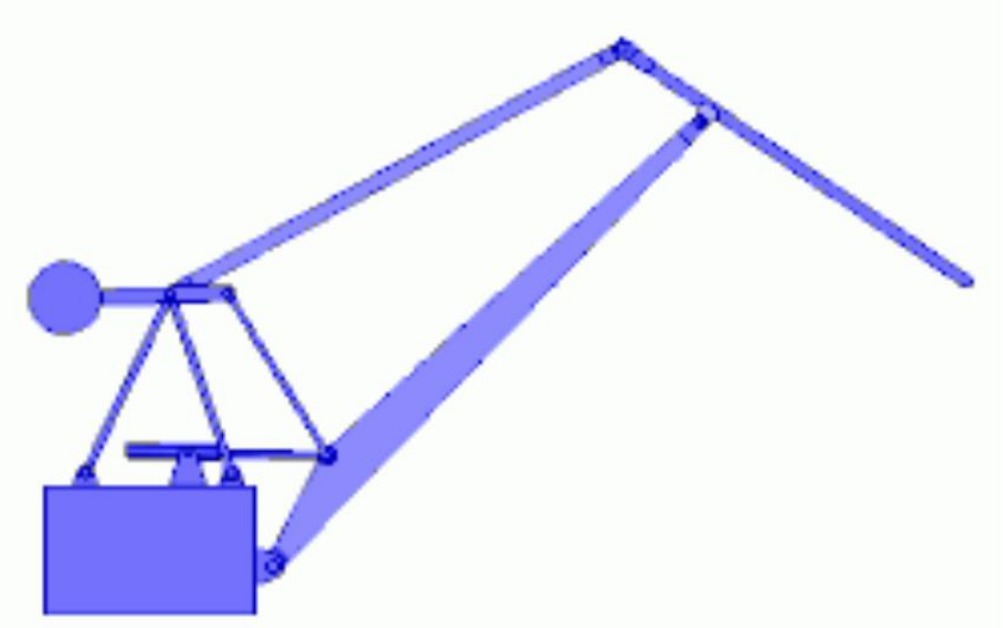
One major modification we made to the ‘level-luffing’ crane design, is the ability for our front arm to dangle as it extends forward. The reason we did this is because we needed to extend the reaching arm to make it lower to the ground, but this caused our claw to clip into the ground. So we allowed the arm to pivot, to keep the claw level

We also removed the counter-weight and added legs for stability.

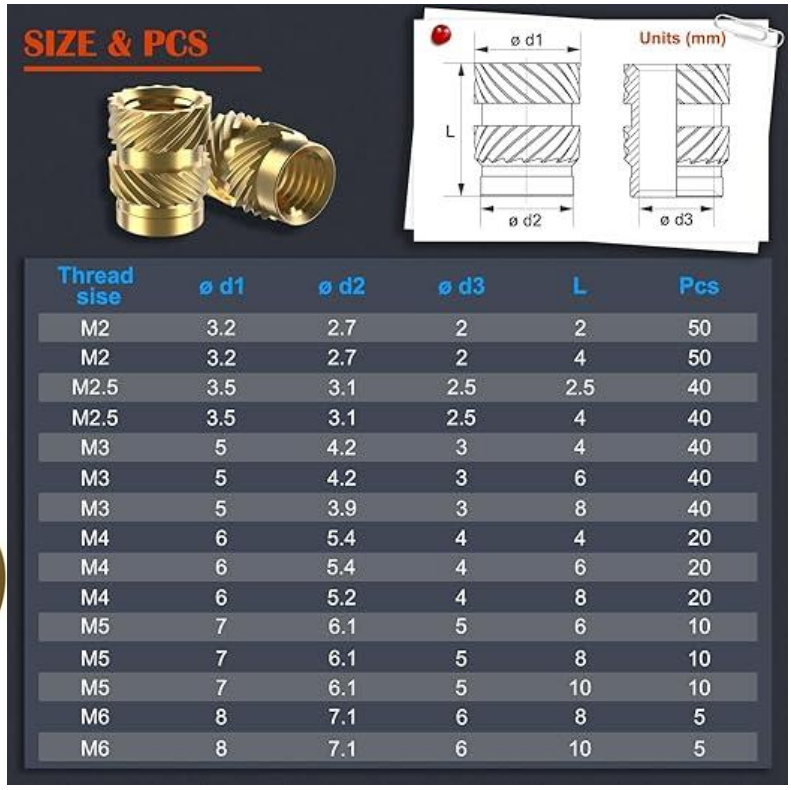
We also modified the base of the crane into a more open, and more cost-effective frame.

https://www.youtube.com/watch?v=goaKXiSi53g





1.3. COTS component



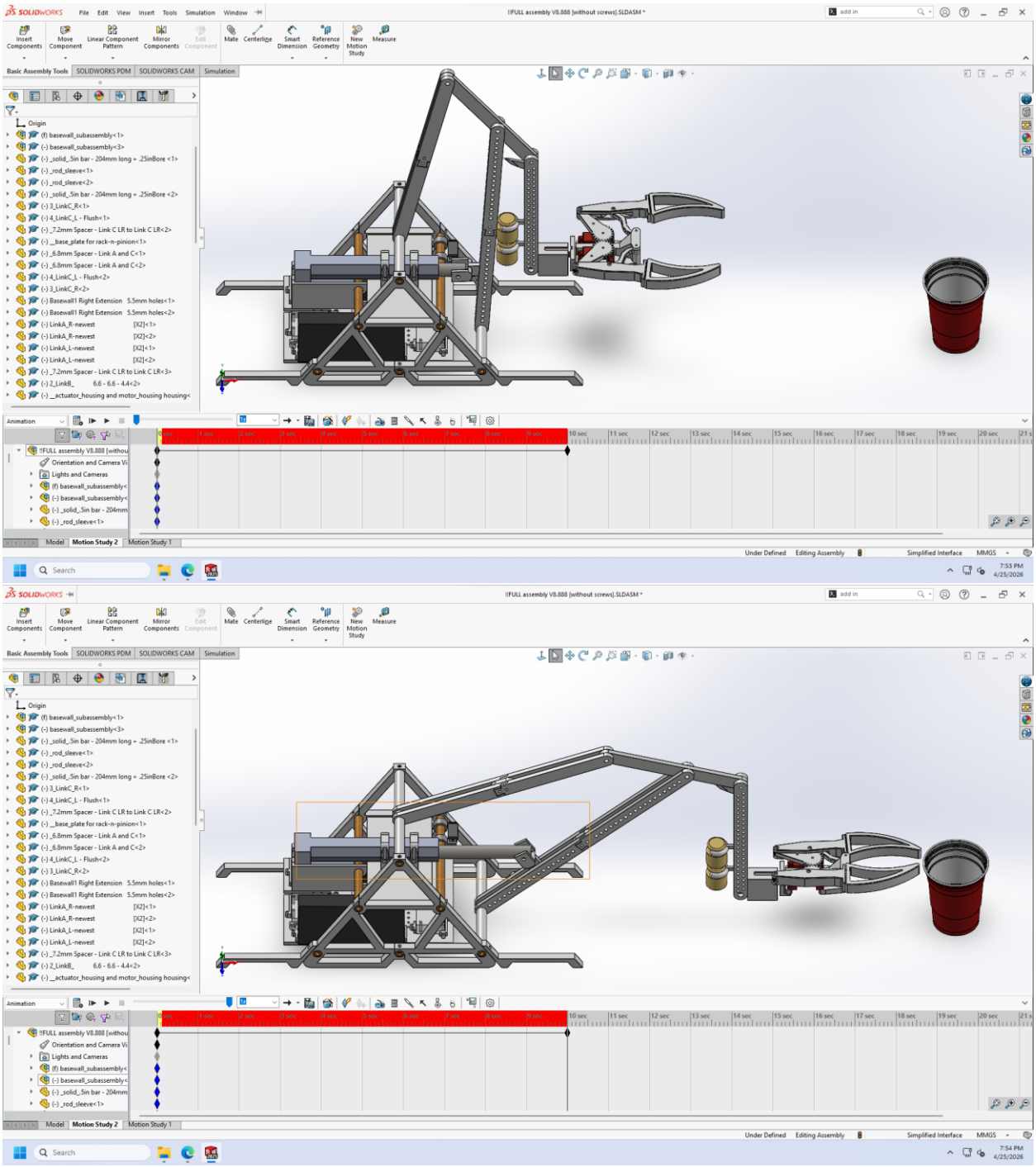
Thread sise	ø d1	ø d2	ø d3	L	Pcs
M2	3.2	2.7	2	2	50
M2	3.2	2.7	2	4	50
M2.5	3.5	3.1	2.5	2.5	40
M2.5	3.5	3.1	2.5	4	40
M3	5	4.2	3	4	40
M3	5	4.2	3	6	40
M3	5	3.9	3	8	40
M4	6	5.4	4	4	20
M4	6	5.4	4	6	20
M4	6	5.2	4	8	20
M5	7	6.1	5	6	10
M5	7	6.1	5	8	10
M5	7	6.1	5	10	10
M6	8	7.1	6	8	5
M6	8	7.1	6	10	5

Used: 4x 6mm long M4 inserts the in claw's grabber's

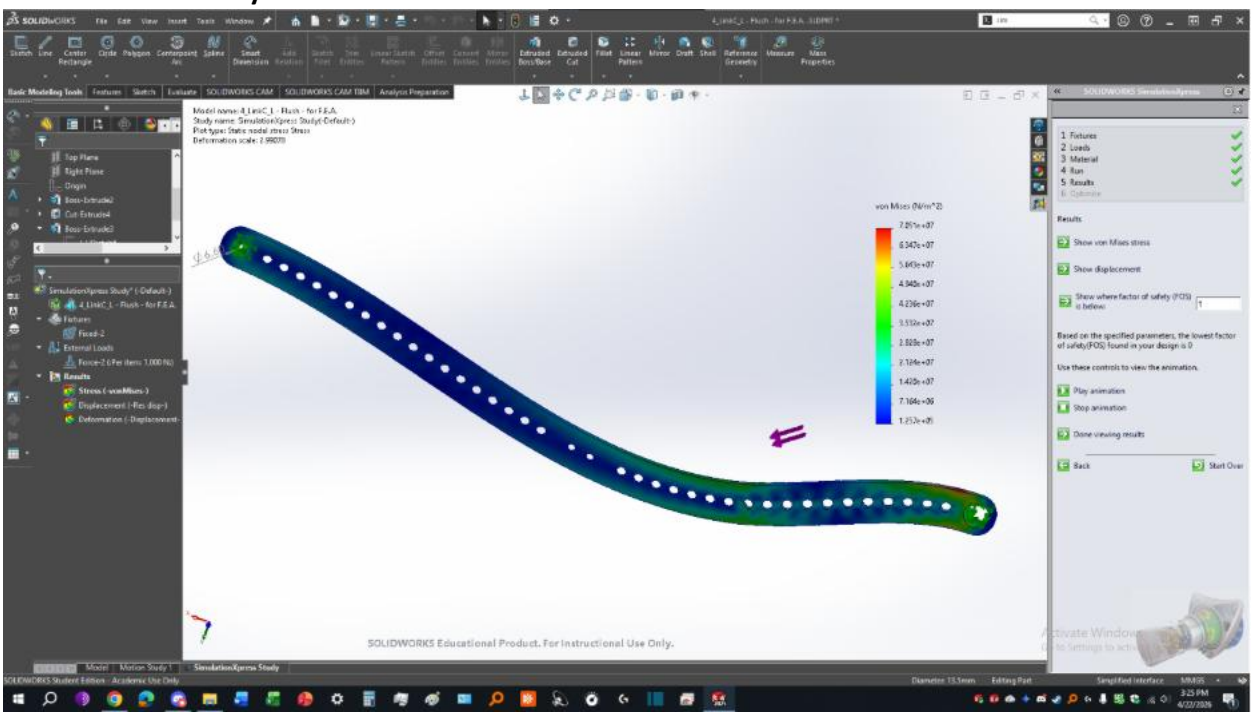
Item: 400Pcs Threaded Inserts, M2 M2.5 M3 M4 M5 M6 Assortment Kit
Cost: $15.98
Manufacturer: Ktehloy [No Official Website]
Distributor: Amazon
Purchase Link: https://a.co/d/04GWM11r

2. SIMULATION

2.1. Animation



2.2. Simulation Analysis



The part was loaded at the loaded at the 11th small hole from the bottom right, similar to how the linear actuator will apply a force to the link at that specific hole.

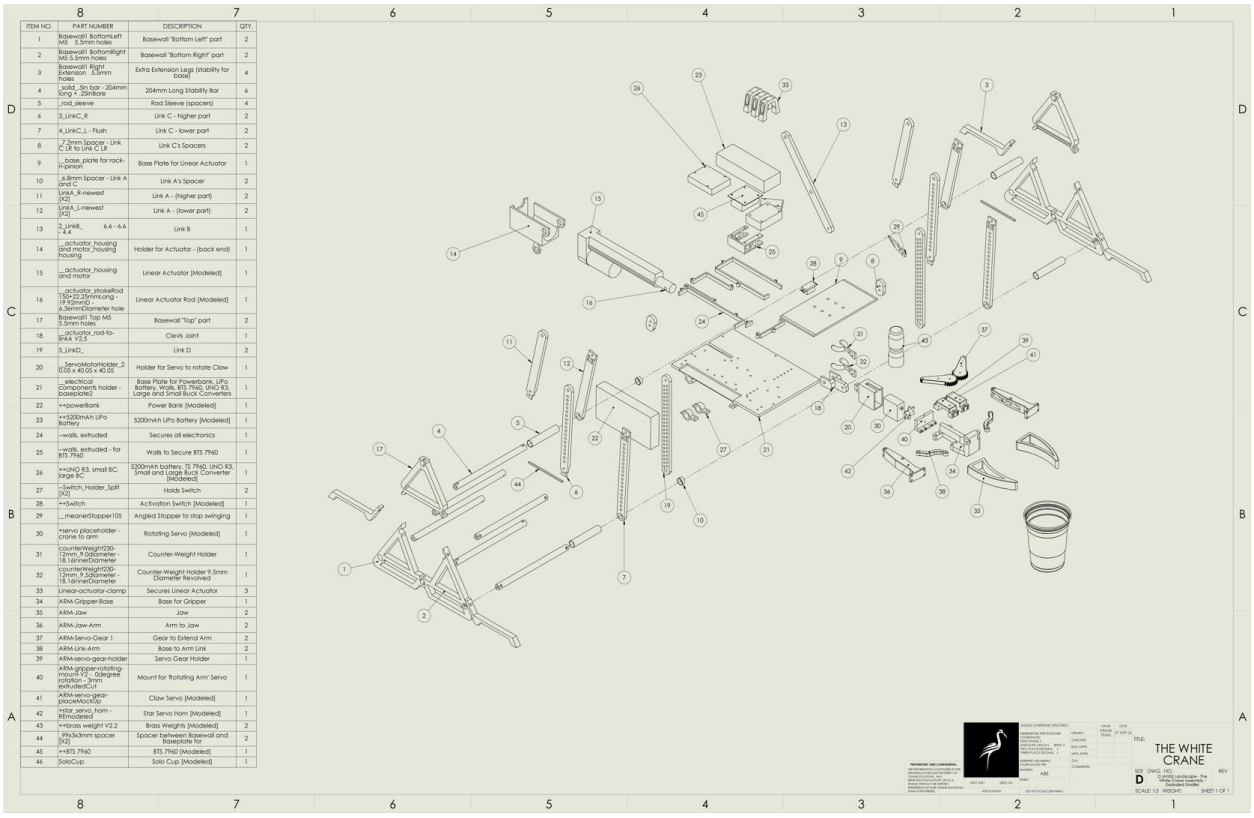
The part had one fixed point at the top left, which connects to a link, and the other fixed point at the bottom right was fixed to the base's rod.

The force we applied was 1000 Newtons, since the maximum force the actuator could produce was 1000 Newtons, and we wanted to see the force at that spot if the whole link is fixed.

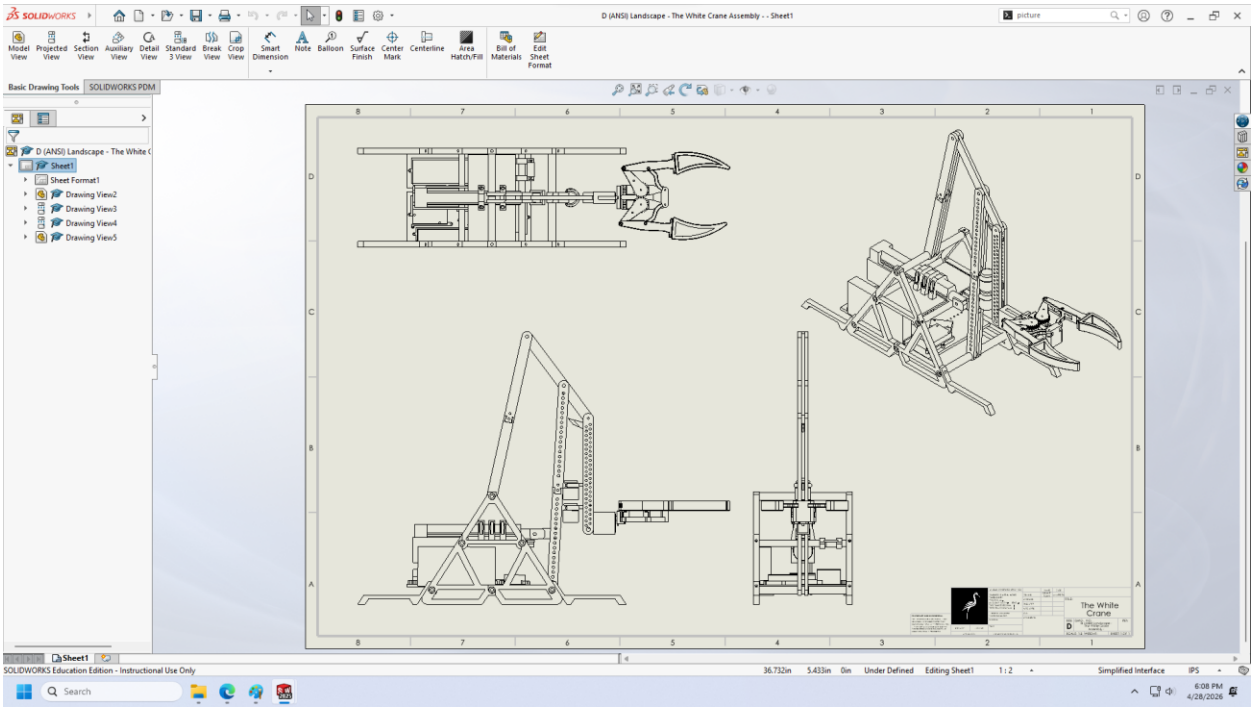
The part that's fixed has red color represents that it needs to be worked on as it experiences the most pressure that may cause it to deform and apart from that the FOI of that part could be changed to reduce the deformation rate. Overall, even though the part experiences a 1000N load it has only the fixed bottom area that experiences maximum load and could potentially deform which could be improved as well. However, the part in general has a lot of blue color that is the lowest stress area along with some greens that has the second highest deformation rate. The Von meses table on the right represents this data.

3. ENGINEERING WORKING DRAWINGS

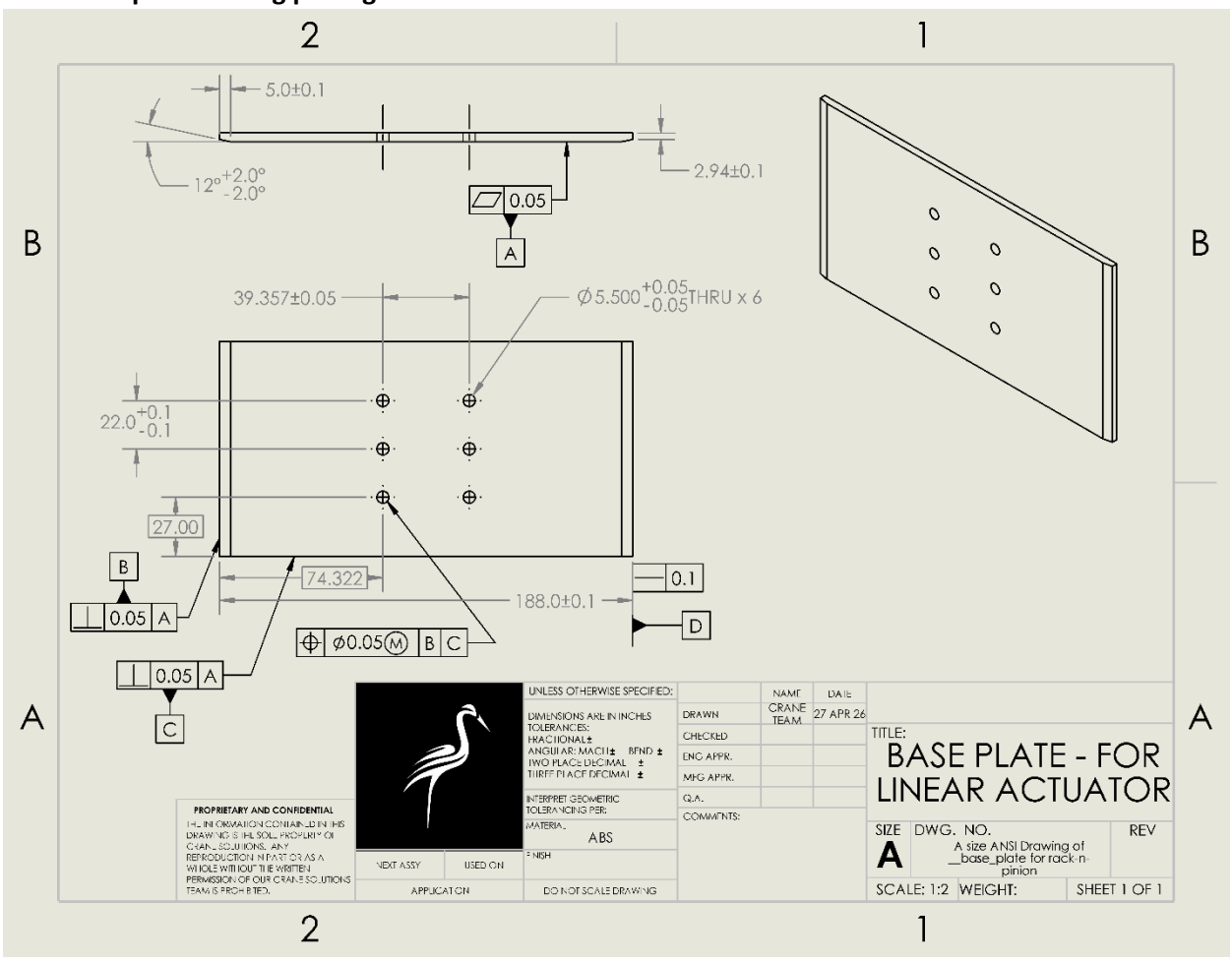
3.1. Assembly drawing



ITEM NO.	PART NUMBER	DESCRIPTION	QTY.
1	Basewall1 BottomLeft M5 - 5.5mm holes	Basewall "Bottom Left" part	2
2	Basewall1 BottomRight M5 5.5mm holes	Basewall "Bottom Right" part	2
3	Basewall1 Right Extension - 5.5mm holes	Extra Extension Legs (stability for base)	4
4	_solid_ Sin bar - 204mm long + .25inBore	204mm Long Stability Bar	6
5	_rod_sleeve	Rod Sleeve (spacers)	4
6	3_LinkC_R	Link C - higher part	2
7	4_LinkC_L - Flush	Link C - lower part	2
8	_7.2mm Spacer - Link C_LR to Link C_LR	Link C's Spacers	2
9	_base_plate for rack-n-pinion	Base Plate for Linear Actuator	1
10	_6.8mm Spacer - Link A's and C	Link A's Spacer	2
11	LinkA_R-newest [X2]	Link A - (higher part)	2
12	LinkA_L-newest [X2]	Link A - (lower part)	2
13	2_LinkB_ 6.6 - 6.6 - 6.4	Link B	1
14	_actuator_housing and motor_housing housing	Holder for Actuator - (back end)	1
15	_actuator_housing and motor	Linear Actuator [Modeled]	1
16	_actuator_strokeRod 130+22.25mmLong - 19.95mmD - 6.36mmDiameter hole	Linear Actuator Rod [Modeled]	1
17	Basewall1 Top M5 5.5mm holes	Basewall "Top" part	2
18	_actuator_rod-to-linkA V2.5	Clevis Joint	1
19	5_LinkD_	Link D	2
20	_ServoMotorHolder_2 0.05 x 40.05 x 40.05	Holder for Servo to rotate Claw	1
21	_electrical Components holder - baseplate2	Base Plate for Powerbank, LiPo Battery, Walls, BTS 7960, UNO R3, Large and Small Buck Converters	1
22	++powerBank	Power Bank [Modeled]	1
23	++5200mAh LiPo Battery	5200mAh LiPo Battery [Modeled]	1
24	—walls_extruded	Secures all electronics	1
25	—walls_extruded - for BTS 7960	Walls to Secure BTS 7960	1
26	++UNO R3, small BC, large BC	5200mAh battery, TS 7960, UNO R3, Small and Large Buck Converter [Modeled]	1
27	-_Switch_Holder_Split [X2]	Holds Switch	2
28	++Switch	Activation Switch [Modeled]	1
29	_menonerStopper105	Angled Stopper to stop swinging	1
30	+servo placeholder - crane to arm	Rotating Servo [Modeled]	1
31	counterWeight230-12mm_9.5diameter - 18.16innerDiameter	Counter-Weight Holder	1
32	counterWeight230-12mm_9.5diameter - 18.16innerDiameter	Counter-Weight Holder 9.5mm Diameter Revolved	1
33	Linear-actuator-clamp	Secures Linear Actuator	3
34	ARM-Gripper-Base	Base for Gripper	1
35	ARM-Jaw	Jaw	2
36	ARM-Jaw-Arm	Arm to Jaw	2
37	ARM-Servo-Gear 1	Gear to Extend Arm	2
38	ARM-Link-Arm	Base to Arm Link	2
39	ARM-servo-gear-holder	Servo Gear Holder	1
40	ARM-gripper-rotating-mount-V2 - 0degree rotation - 3mm extrudedCut	Mount for 'Rotating Arm' Servo	1
41	ARM-servo-gear-placeMockUp	Claw Servo [Modeled]	1
42	+star_servo_horn - REmodeled	Star Servo Horn [Modeled]	1
43	++brass weight V2.2	Brass Weights [Modeled]	2
44	_99x3x3mm spacer [X2]	Spacer between Basewall and Baseplate for	2
45	++BTS 7960	BTS 7960 [Modeled]	1
46	SoloCup	Solo Cup [Modeled]	1



3.2. Detail part drawing package #1



3.3. Detail part drawing package #2